CHAIN MATRIX MULTIPLICATION USING WITH DYANMIC PROGRAMMING

```
import time

def matrix_chain(p):
    n = len(p) - 1

    # Create DP table
    dp = [[0] * n for _ in range(n)]

    # Chain length
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float('inf')

            for k in range(i, j):
                cost = (dp[i][k] +
                        dp[k + 1][j] +
                        p[i] * p[k + 1] * p[j])

                if cost < dp[i][j]:
                    dp[i][j] = cost

    return dp[0][n - 1]


# User input
n = int(input("Enter number of matrices: "))

p = []

print("Enter dimensions:")

for i in range(n):
    if i == 0:
        rows = int(input("Enter rows of Matrix 1: "))
    else:
        rows = p[-1]

    cols = int(input(f"Enter columns of Matrix {i + 1}: "))

    if i > 0 and rows != p[-1]:
        print("Invalid dimensions!")
        exit()

    p.append(rows)
    p.append(cols)
```

```
# Remove duplicate dimensions
dimensions = [p[0]]
for i in range(1, len(p), 2):
    dimensions.append(p[i])

# Start time
start = time.time()

# Dynamic Programming
result = matrix_chain(dimensions)

# End time
end = time.time()

print("\nMinimum number of scalar multiplications:", result)
print("Execution Time:", end - start, "seconds")

print("\nTime Complexity:")
print("Best Case    : O(n^3)")
print("Average Case : O(n^3)")
print("Worst Case   : O(n^3)")

print("Space Complexity: O(n^2)")
```

```
Enter number of matrices: 5
Enter dimensions:
Enter rows of Matrix 1: 3
Enter columns of Matrix 1: 3
Enter columns of Matrix 2: 3
Enter columns of Matrix 3: 2
Enter columns of Matrix 4: 4
Enter columns of Matrix 5: 4

Minimum number of scalar multiplications: 96
Execution Time: 9.441375732421875e-05 seconds

Time Complexity:
Best Case    : O(n^3)
Average Case : O(n^3)
Worst Case   : O(n^3)
Space Complexity: O(n^2)
```